# Generative AI and Its Applications

## Unit 3

(UE23CS342BA4)

## ADVANCED RETRIEVAL-AUGMENTED GENERATION

**Dr. Pooja Agarwal**

Dept. of CSE, PES University, Bangalore

## Acknowledgement

The slides are prepared from various resources from the Universities from abroad and India. Also, some material is taken from reliable resources from internet throughout this course.

# What is RAG?

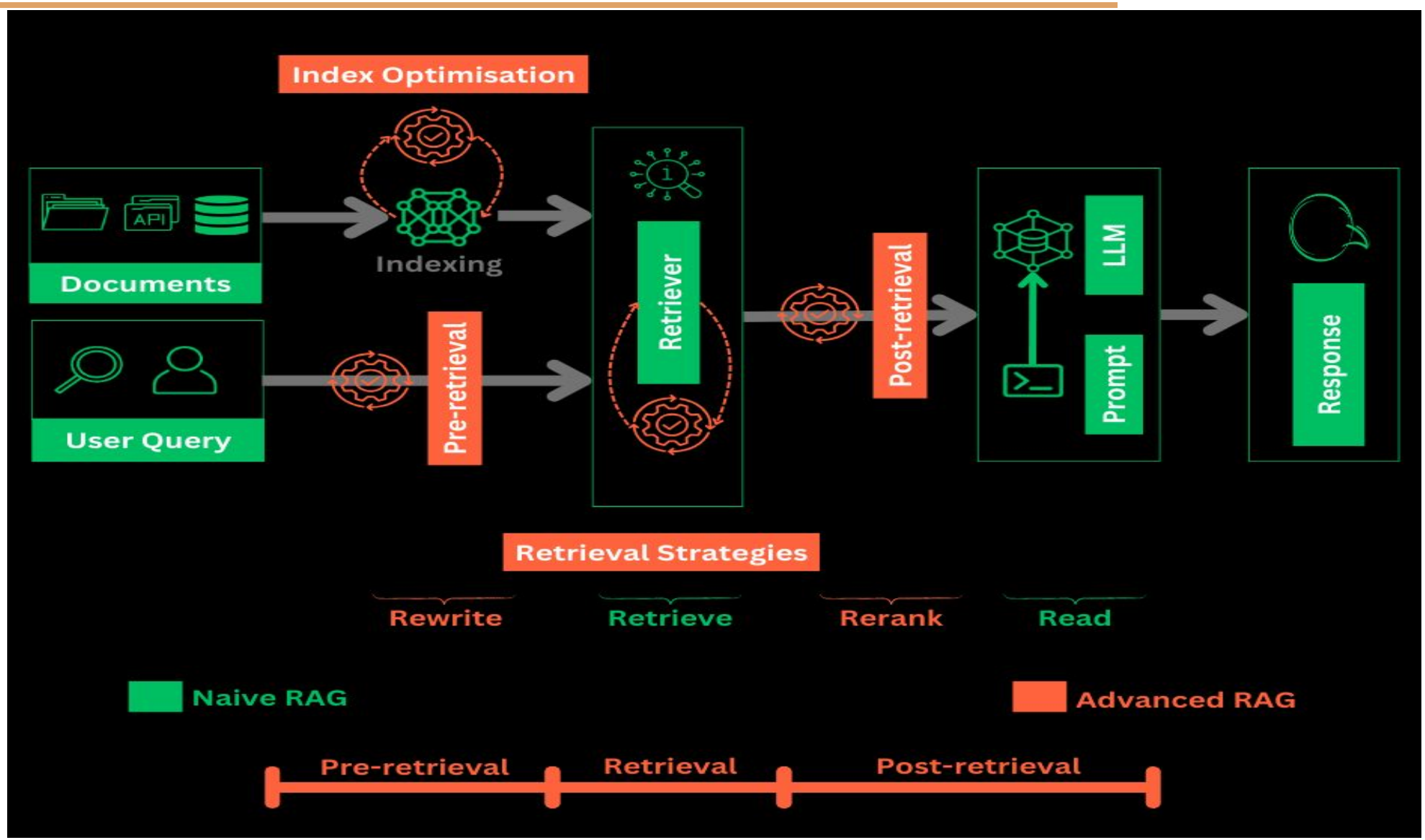
**Basic RAG:** Ingestion, Retrieval, Synthesis pipeline
**Retrieval Algorithms:** FLAT (brute-force), IVF (clustering), HNSW (graph-based), PQ (compression)
**Ranking:** BM25 for sparse retrieval, Cosine Similarity for dense retrieval

## The Problem with Naive RAG

| Issue | Description | Analogy |
|---|---|---|
| **Irrelevant Retrieval** | Top-k chunks often miss the mark | Searching for "apple" and getting fruit recipes when you wanted Apple Inc. stock prices |
| **Context Overload** | Too much irrelevant context confuses the LLM | Giving a student 10 textbooks to answer one simple question |
| **Lost in the Middle** | LLMs focus on start/end of context, ignore middle | Reading only first and last paragraphs of a long article |
| **Hallucinations** | LLM ignores context, invents answers | Student writing answers from memory instead of using provided notes |
| **Latency Issues** | Retrieval from millions of vectors is slow | Finding a needle in a haystack by looking at every straw |

# GenerativeAI and Its Applications

## Quick Recap



# EVOLUTION

**Level 1: Naive RAG**

Query → Retriever → Generator → Response
(Simple pipeline, no optimizations)

**Level 2: Advanced RAG**

Query → Query Transformation → Multi-Stage Retrieval → Reranking → Generator → Response
(Optimized retrieval, multiple enhancements)

**Level 3: Modular RAG**

Query → Router → Multiple Retrievers → Fusion → Self-Correction → Generator
(Dynamic routing, iterative refinement)

# Advanced RAG (Retrieval-Augmented Generation)

**Advanced RAG** is an improved version of the traditional Retrieval-Augmented Generation system that enhances how information is **retrieved, filtered, and used** before generating responses with an LLM. It introduces **multiple optimization techniques** to improve **accuracy, relevance, and reasoning**.

# NEED FOR ADVANCED RAG?

**1. Precision vs. Recall Trade-off**
**Naive RAG:** High recall (finds many relevant docs), low precision (too much noise)
**Advanced RAG:** Balanced precision-recall through multi-stage filtering
**2. Query Complexity**
Real user queries are ambiguous, multi-intent, or poorly phrased
Need query understanding before retrieval
**3. Scale Requirements**
Production systems: Millions/Billions of documents
Need efficient indexing and retrieval (IVF, HNSW you learned)
**4. Cost Optimization**
LLM calls are expensive ($/token)
Better context = fewer tokens = lower cost
**5. User Expectations**
Users expect ChatGPT-like experience with factual accuracy
Need citations, verifiable answers

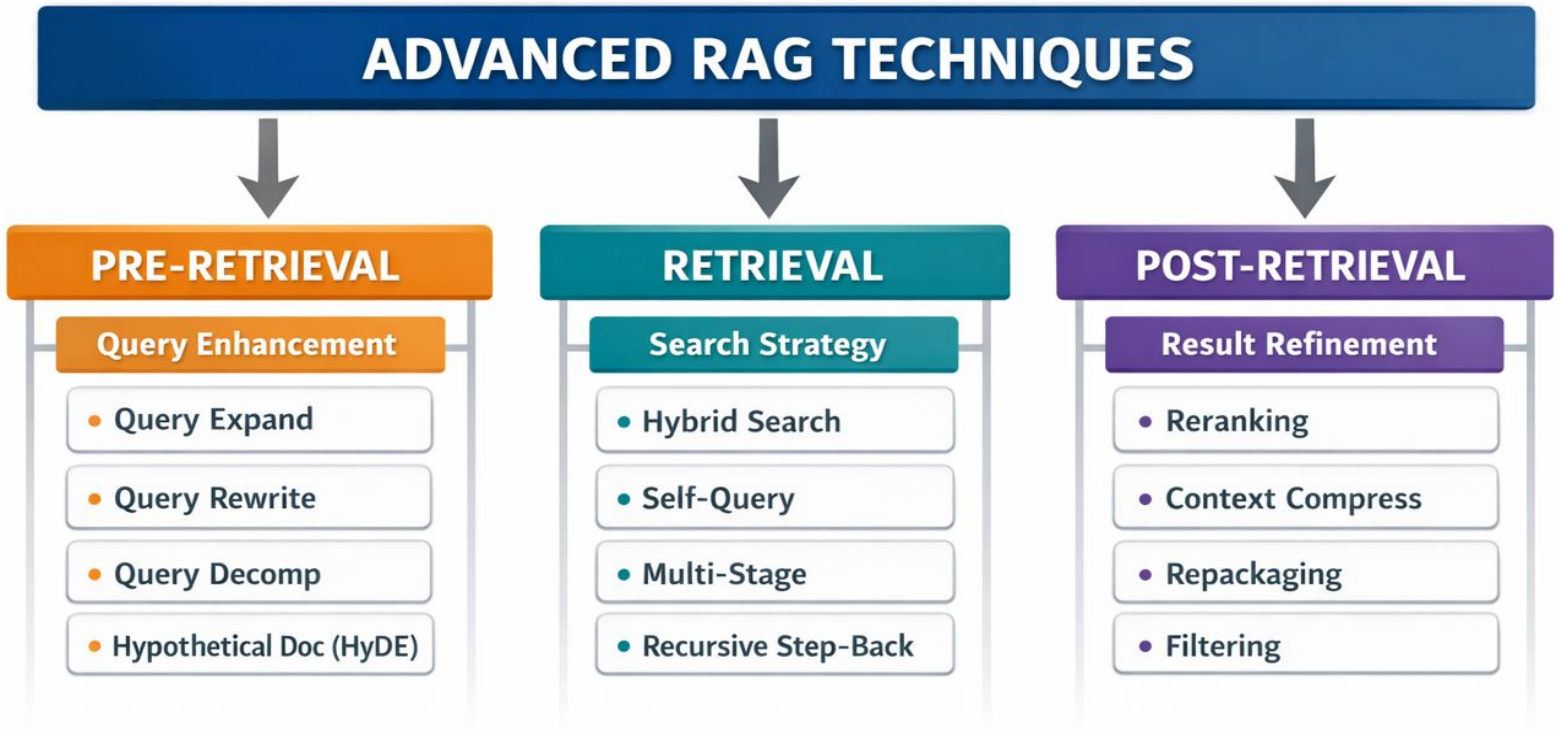
| Problem | Explanation |
|---|---|
| Irrelevant documents | Retriever returns noisy chunks |
| Poor query understanding | User queries may be vague |
| Limited reasoning | LLM receives incomplete context |
| Scalability issues | Large document collections |

# GenerativeAI and Its Applications

## Advanced RAG techniques flowchart diagram

PIPELINE:

User Query
↓
Query Transformation
↓
Multi-Stage Retrieval
↓
Reranking / Filtering
↓
Context Selection
↓
LLM Generation
↓
Final Response



## Query Transformation

The query is modified or expanded before retrieval.
Examples:

- Query rewriting
- Query decomposition
- Step-back prompting

Example:
User Query
"Impact of AI on healthcare"
Rewritten queries:

- AI applications in healthcare
- Benefits of AI in medical diagnosis
- AI-based medical systems

## Key Techniques in Advanced RAG

Hybrid Retrieval Combines **two types of search**.

| Method | Purpose |
|---|---|
| Sparse Retrieval (BM25) | Keyword matching |
| Dense Retrieval (Embeddings) | Semantic similarity |

This improves **recall and precision**.

## Key Techniques in Advanced RAG

**Hybrid Retrieval** Combines **two types of search**.

| Method | Purpose |
|---|---|
| Sparse Retrieval (BM25) | Keyword matching |
| Dense Retrieval (Embeddings) | Semantic similarity |

This improves **recall and precision**.

**Multi-Stage Retrieval**

Retrieval happens in **multiple filtering stages**.
Stage 1 → Fast retrieval (BM25 / ANN)
Stage 2 → Reranking using cross-encoder
Stage 3 → Select best chunks
This reduces **noise in retrieved documents**.

Key Techniques in Advanced RAG

## Reranking

Retrieved documents are **rescored using better models**.
Examples:
Cross-encoder models
LLM rerankers
Cohere reranker
Purpose: Select the **most relevant top-k documents**.
.

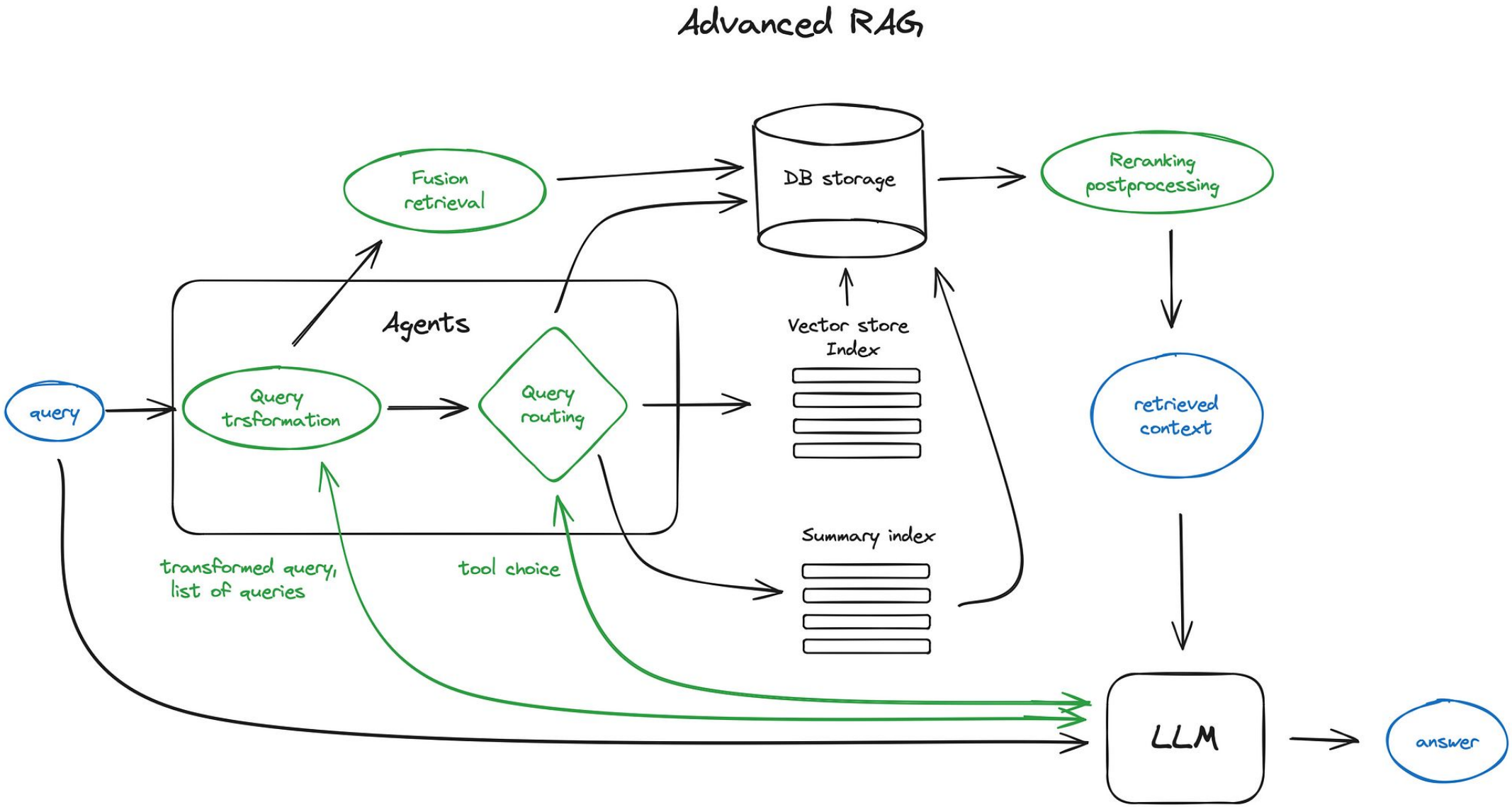
## Context Enrichment

After retrieval, additional context is added.
Examples:
Sentence window retrieval
Parent-child chunk retrieval
Auto-merging retriever
This improves **LLM reasoning ability**.

## Recursive Retrieval

The system retrieves documents **iteratively**.
Query → Retrieve documents
↓
Extract entities
↓
Generate new queries
↓
Retrieve more documents
Used in **complex knowledge systems**.

## The Advanced RAG Pipeline - High Level



# GenerativeAI and Its Applications

**Chunking:**

- Transformer models have fixed input sequence length and even if the input context window is large, the vector of a sentence or a few better represents their semantic meaning than a vector averaged over a few pages of text (depends on the model too, but true in general).
- **Chunk your data** is spliting the initial documents in chunks of some size without loosing their meaning (splitting your text in sentences or in paragraphs, not cutting a single sentence in two parts). There are various text splitter implementations capable of this task.

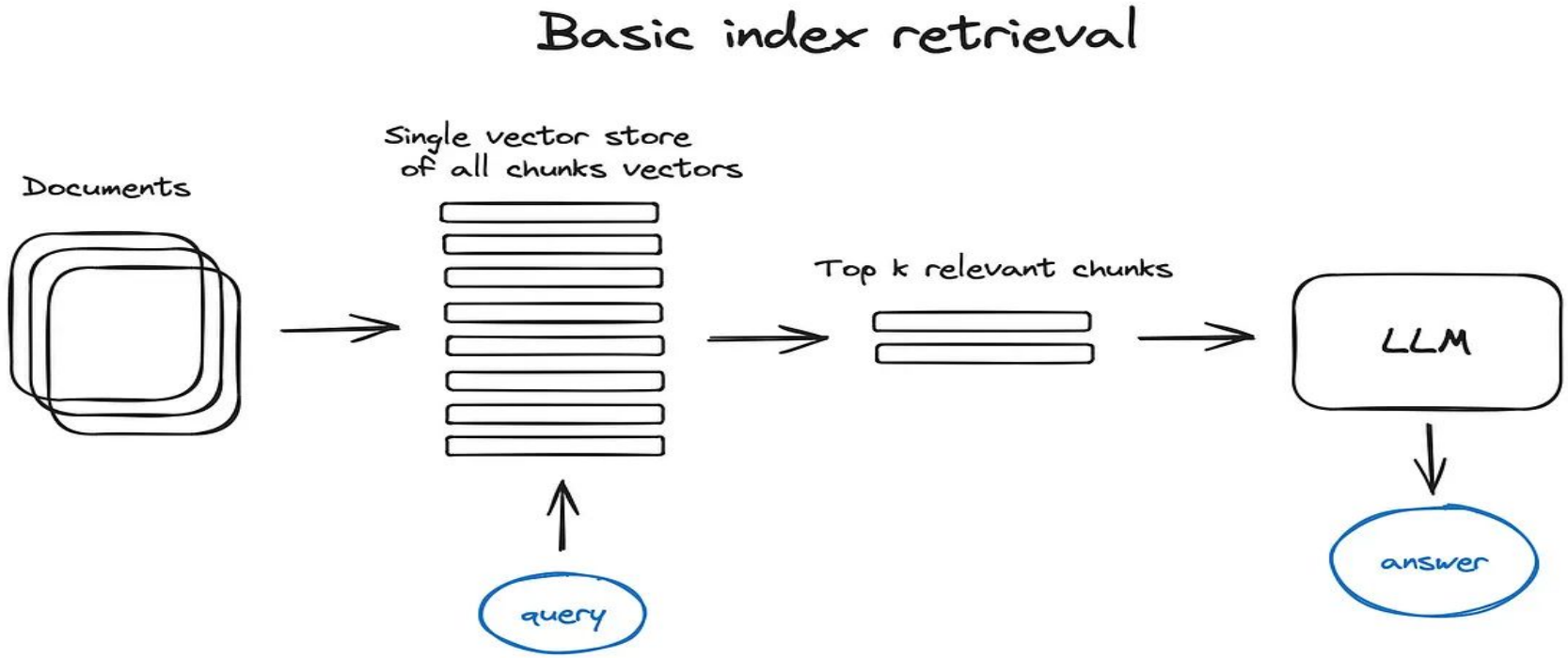
**Consider:** Encoder models like BERT-based Sentence Transformers take 512 tokens at most, OpenAI ada-002 is capable of handling longer sequences like 8191 tokens, but the compromise here is enough context for the LLM to reason upon vs specific enough text embedding in order to efficiently execute search upon.

- The next step is to choose a **model to embed our chunks. The search optimised models** like bge-large or E5 embeddings family can be chosen.

# GenerativeAI and Its Applications

## Search index- Vector store index

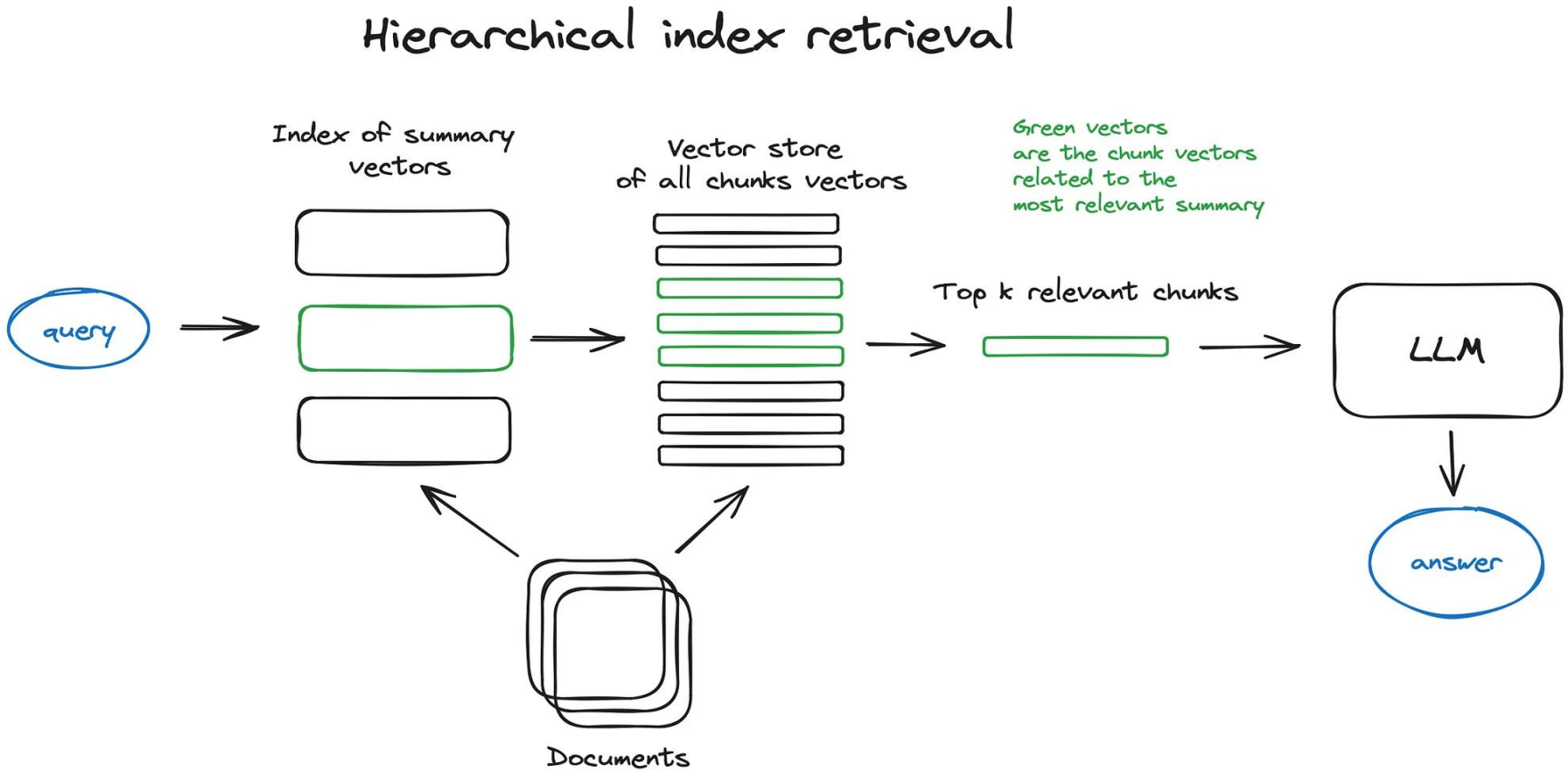
- The crucial part of the RAG pipeline is the search index, storing your vectorised content we got in the previous step. The most naive implementation uses a flat index which is a brute force distance calculation between the query vector and all the chunks' vectors.
- A proper search index, optimised for efficient retrieval on 10000+ elements scales is a vector index like faiss, nmslib or annoy, using some Approximate Nearest Neighbours implementation like clustring, trees or HNSW algorithm.
- Depending on your index choice, data and search needs you can also store metadata along with vectors and then use metadata filters to search for information within some dates or sources for example.



# GenerativeAI and Its Applications

## Search index: Hierarchical indices

- In case you have many documents to retrieve from, you need to be able to efficiently search inside them, find relevant information and synthesise it in a single answer with references to the sources.
- An efficient way to do that in case of a large database is to **create two indices: one composed of summaries and the other one composed of document chunks**, and to search in two steps, first filtering out the relevant docs by summaries and then searching just inside this relevant group.



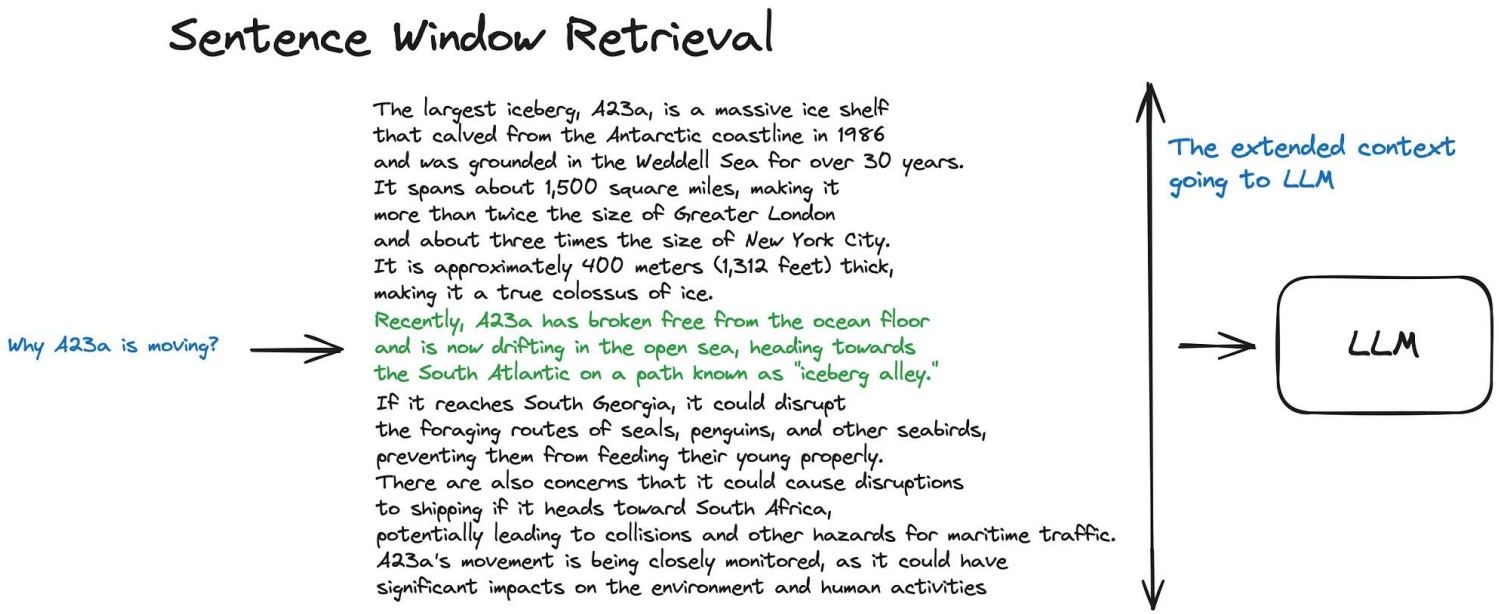
## Search index- Hypothetical Questions and HyDE

- Another approach is to ask an LLM to **generate a question for each chunk and embed these questions in vectors**, at runtime performing query search against this index of question vectors (replacing chunks vectors with questions vectors in our index) and then after retrieval route to original text chunks and send them as the context for the LLM to get an answer.
- This approach improves search quality due to a **higher semantic similarity between query and hypothetical question** compared to what we'd have for an actual chunk.
- There is also the reversed logic apporach called HyDE — you ask an LLM to generate a hypothetical response given the query and then use its vector along with the query vector to enhance search quality.

## Search index- Context enrichment

- **The concept here is to retrieve smaller chunks for better search quality, but add up surrounding context for LLM to reason upon.**
- There are two options — to expand context by sentences around the smaller retrieved chunk or to split documents recursively into a number of larger parent chunks, containing smaller child chunks.

### 1.Sentence Window Retrieval

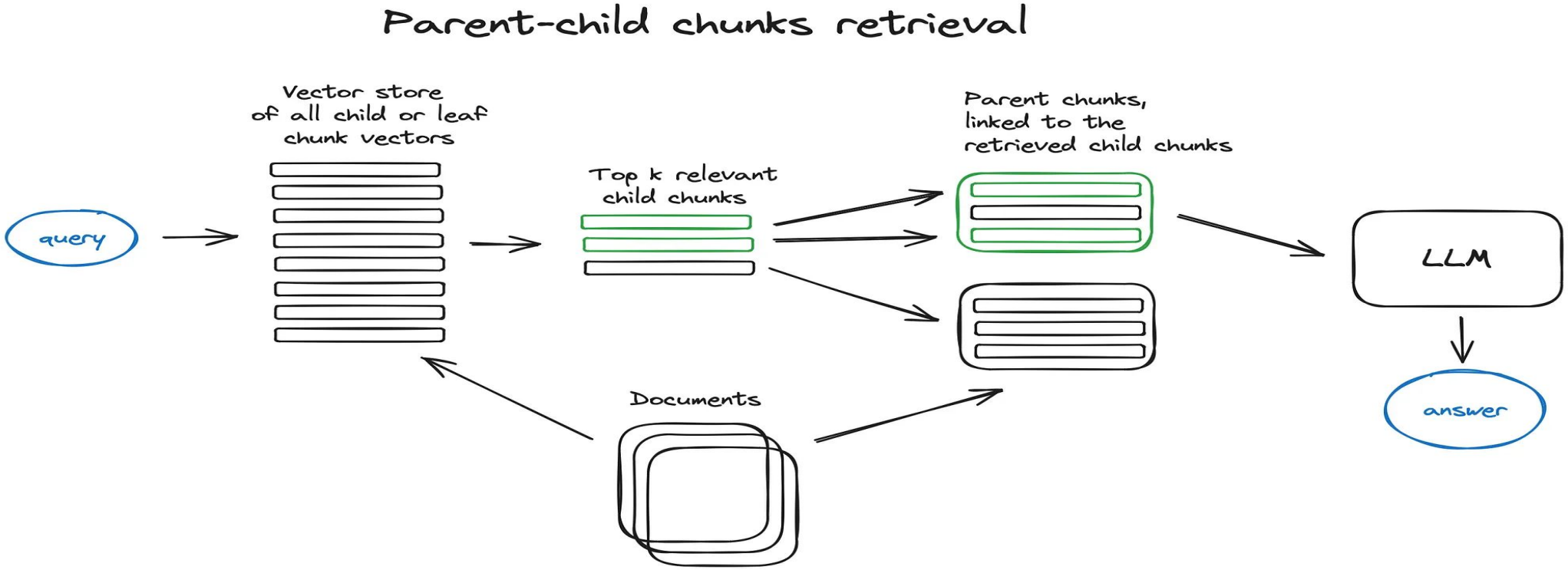



- In this scheme each sentence in a document is embedded separately which provides great accuracy of the query to context cosine distance search.
- In order to better reason upon the found context after fetching the most relevant single sentence we extend the context window by *k* sentences before and after the retrieved sentence and then send this extended context to LLM.

# GenerativeAI and Its Applications

**Search index- Context enrichment** **2. Auto-merging Retriever (aka Parent Document Retriever)**

- The idea here is pretty much similar to Sentence Window Retriever: to search for more granular pieces of information and then to extend the context window before feeding said context to an LLM for reasoning. Documents are split into smaller child chunks referring to larger parent chunks.

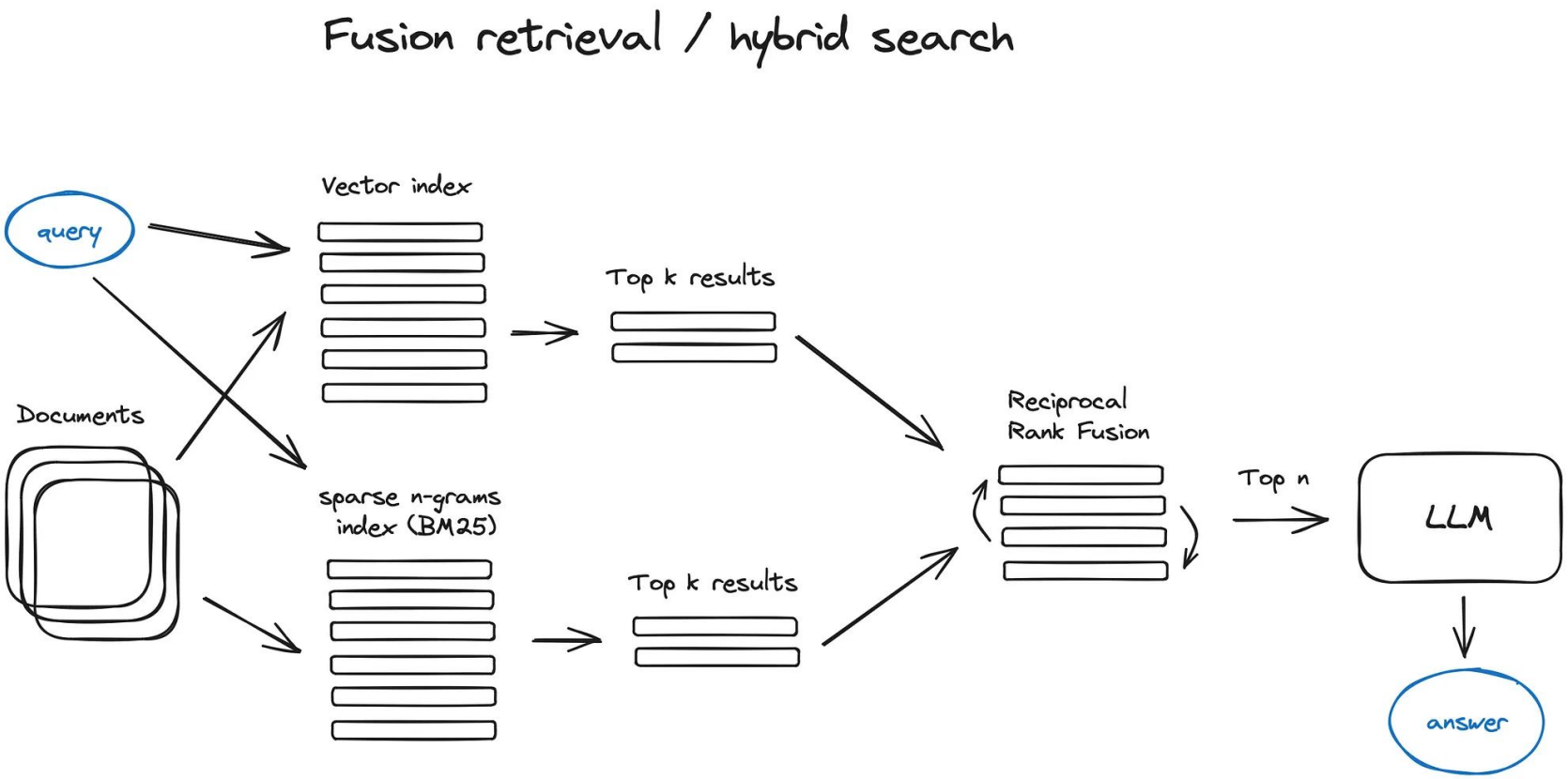



- Fetch smaller chunks during retrieval first, then if more than *n* chunks in top *k* retrieved chunks are linked to the same parent node (larger chunk)
- We replace the context fed to the LLM by this parent node — works like auto merging a few retrieved chunks into a larger parent chunk, hence the method name.

# GenerativeAI and Its Applications

## Fusion retrieval or hybrid search

- To properly combine the retrieved results with different similarity scores — this problem is usually solved with the help of the Reciprocal Rank Fusion algorithm, reranking the retrieved results for the final output.



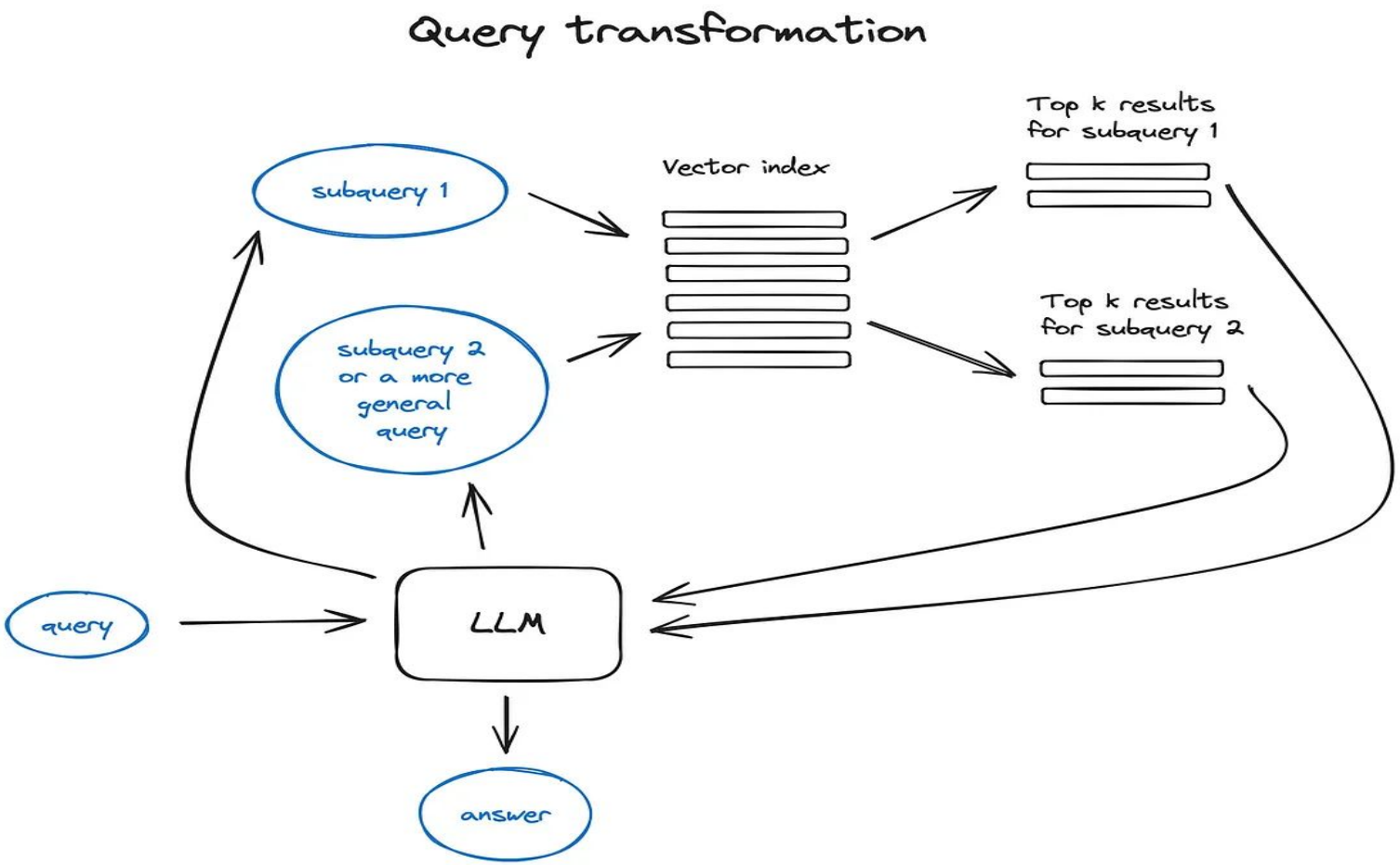

- Hybrid or fusion search usually provides better retrieval results as two complementary search algorithms are combined, taking into account both semantic similarity and keyword matching between the query and the stored documents.

## Reranking & filtering

- Now we got our retrieval results with any of the algorithms described above, it is time to refine them through filtering, re-ranking or some transformation.
- In LlamaIndex there is a variety of available **Postprocessors**, **filtering out results based on similarity score, keywords, metadata or reranking them with other models** like an LLM, sentence-transformer cross-encoder, Cohere reranking endpoint or based on metadata like date recency — basically, all you could imagine.
- This is the final step before feeding our retrieved context to LLM in order to get the resulting answer.

# GenerativeAI and Its Applications

## Query transformations




**If the query is complex, LLM can decompose it into several sub queries.**

Anology:

if you ask:

— *"What framework has more stars on Github, Langchain or LlamaIndex?",*

— *"How many stars does Langchain have on Github?"*

— *"How many stars does Llamaindex have on Github?"*

They would be executed in parallel and then the retrieved context would be combined in a single prompt for LLM to synthesize a final answer to the initial query.

# USE CASES OF ARAG?

| Company | Use Case | Naive RAG Problem | Advanced RAG Solution | Company |
|---|---|---|---|---|
| **JPMorgan** | Financial Document Analysis | Missed nuanced regulatory requirements | Query expansion + Multi-stage retrieval | **JPMorgan** |
| **Merck** | Medical Literature Q&A | Retrieved irrelevant drug studies | Reranking + Domain-specific embeddings | **Merck** |
| **Uber** | Customer Support | High latency, irrelevant answers | Hybrid search + Caching | **Uber** |
| **Notion** | AI Q&A over notes | Couldn't handle complex queries | Query rewriting + Recursive retrieval | **Notion** |

# WHAT IS RETREIVAL ENHANCEMENT ?

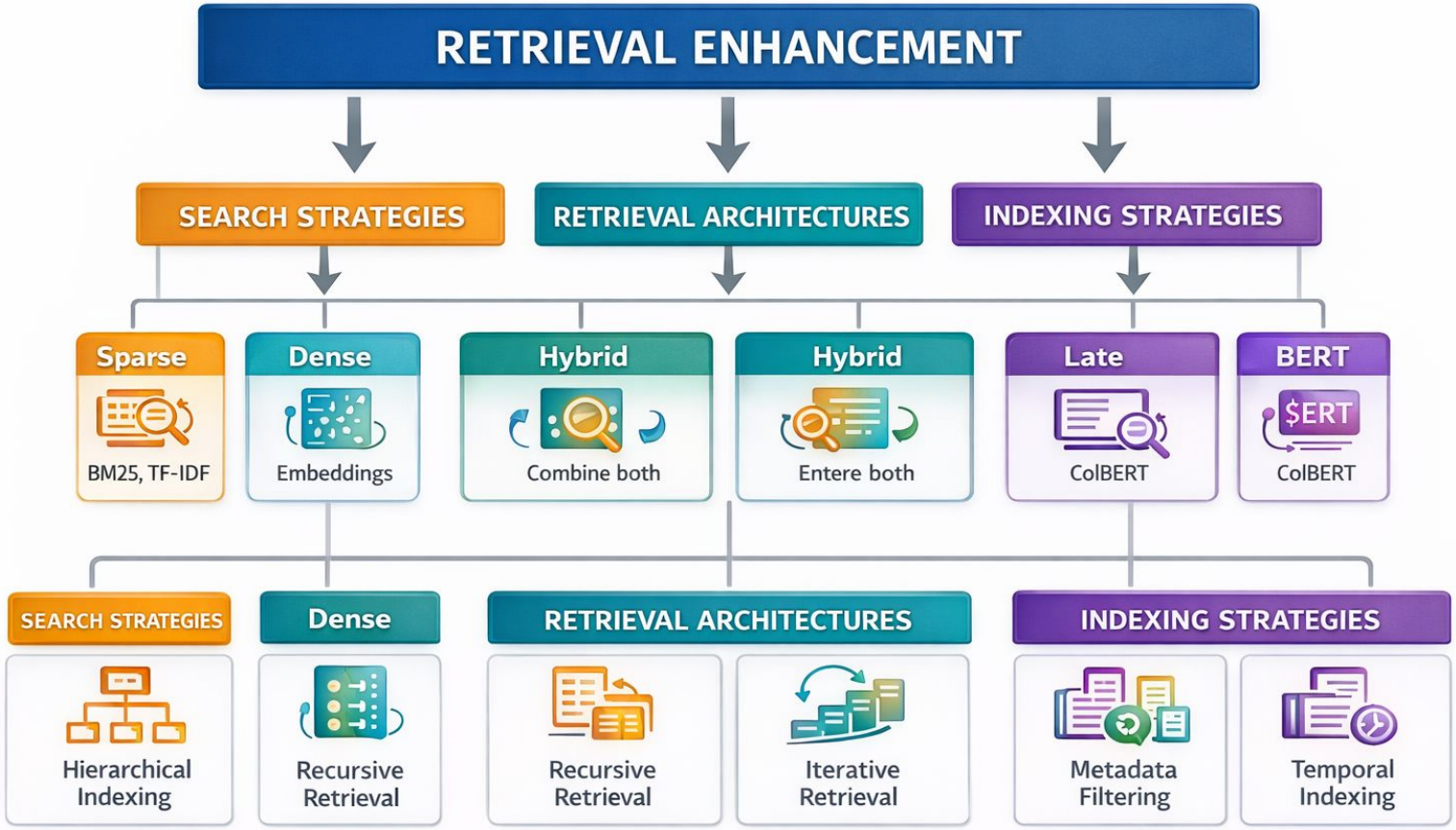
A set of techniques applied during the retrieval phase to improve the quality, relevance, and comprehensiveness of retrieved documents before they reach the LLM.

Retrieval Enhancement Techniques are methods used to improve the quality, relevance, and accuracy of information retrieved in Retrieval-Augmented Generation (RAG) systems. In traditional retrieval systems, the model searches for documents based on a simple query, which may lead to irrelevant or incomplete results. Retrieval enhancement techniques address this limitation by improving how queries are processed and how documents are searched and selected.

Why Retrieval Matters in RAG

| Issue | Example |
|---|---|
| Hallucination | LLM invents facts |
| Outdated knowledge | Model trained in 2023 |
| No private data | Company docs unavailable |

# RETREIVAL ENHANCEMENT Landscape?



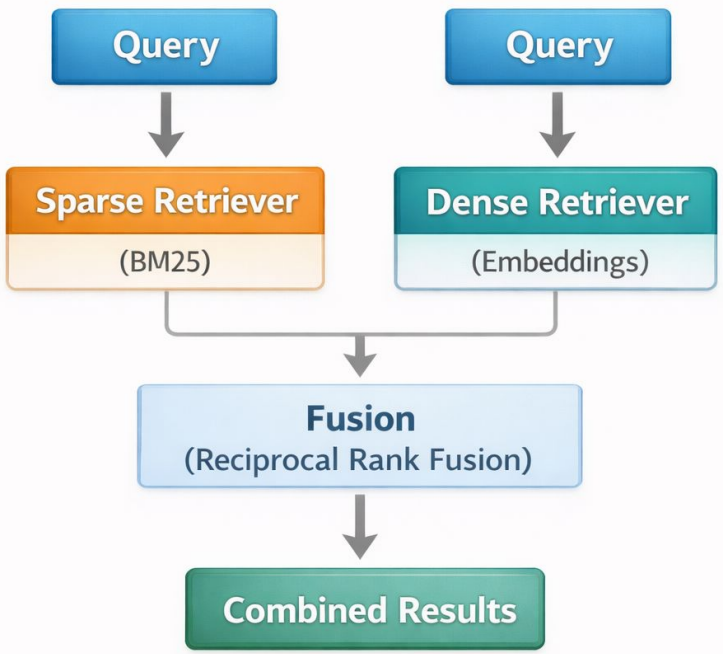
## Sparse vs Dense Retrieval -

**Sparse Retrieval (BM25, TF-IDF):**
[cat, sat, on, mat] → {cat:1, sat:1, on:1, mat:1}
(Bag of words, high-dimensional sparse vector)
**Based on:** Exact keyword matching
**Strength:** Precision, interpretability
**Weakness:** Vocabulary mismatch ("car" vs "automobile")
**Analogy:** Library card catalog - looks for exact book titles

**Dense Retrieval (Embeddings)**
[cat, sat, on, mat] → [0.23, -0.45, 0.67, ...] (dense vector)
**Based on:** Semantic similarity
**Strength:** Understands concepts, synonyms
**Weakness:** Can miss exact matches, less interpretable
**Analogy:** Librarian who understands concepts - finds books about "felines" when you ask for "cats"

| Feature | Sparse Retrieval | Dense Retrieval |
|---|---|---|
| Representation | Bag of Words | Embeddings |
| Example | BM25 | BERT / E5 |
| Strength | Exact matches | Semantic similarity |
| Weakness | Vocabulary mismatch | May miss keywords |

# Hybrid retrieval flowchart process?




**Why Hybrid?**

- BM25 catches **exact terms** (product codes, names)
- Embeddings catch **semantic meaning** (concepts, synonyms)
- Together it is a comprehensive retrieval

Hybrid scoring:

```
Final Score = RRF(BM25 rank) + RRF(Dense rank)

If document ranks high in both → very relevant
```

# Multi-Stage Retrieval process?

Stage 1: Coarse Retrieval
│ Query → IVF/HNSW → Top 100 candidates
│ (Fast, approximate, high recall)
│
Stage 2: Fine Re-ranking
│ Top 100 → Cross-encoder → Top 10 candidates
│ (Slower, precise, high precision)
│
Stage 3: Context Selection
│ Top 10 → Deduplication → Final 5 chunks
│ (Quality control)

Analogy –

1.)Medical Diagnosis:

Stage 1 (Triage Nurse): Quick assessment → 100 possible conditions

Stage 2 (General Doctor): Detailed check → 10 likely conditions

Stage 3 (Specialist): Final diagnosis → 1 confirmed condition

2.)Google Search Pipeline

1. BM25 retrieves 1000 docs
2. BERT reranker scores them
3. Top 10 results shown

# Recursive Retrieval Retrieval process?

Retrieve → Extract entities → Retrieve more based on entities → Repeat

Query: "What were the side effects of the COVID vaccine trials in 2020?"
Round 1:
── Retrieve: Documents about COVID vaccines
── Extract: "Pfizer", "Moderna", "Phase 3 trials", "adverse events"
── Identify gaps: Need specific trial data

Round 2:
── New queries: "Pfizer Phase 3 adverse events 2020", "Moderna clinical trial safety data"
── Retrieve: Specific trial documents
── Identify gaps: Need regulatory reports

Round 3:
── New query: "FDA adverse event reporting system COVID 2020"
── Retrieve: Regulatory safety reports
Final: Combine all retrieved context

Query
↓
Retrieve docs
↓
Extract entities
↓
New queries
↓
Retrieve again

# Step-Back Retrieval Retrieval process?

**The Problem:** LLMs often fail on questions requiring high-level reasoning

Original Query: "What caused the temperature anomaly in February 2023?"

Step 1: Step-back to concept
└── "What factors influence temperature variations?"

Step 2: Retrieve general principles
└── Documents about climate patterns, El Niño, greenhouse effects

Step 3: Original retrieval
└── Specific documents about Feb 2023 weather data

Step 4: Combine both
└── General principles + Specific data → Better reasoning

**Why does this work:**
Original query = specific facts
Step-back query = conceptual knowledge
LLM needs both for reasoning
LLM generates a higher-level query

# Self-Querying Retrieval Retrieval process?

LLM extracts metadata filters from natural language query

Query: "Show me documents about machine learning published by Google in 2023"

Step 1: LLM parses query into:

```
{
  "query": "machine learning",
  "filter": {
    "author_org": "Google",
    "year": {"$gte": 2023, "$lt": 2024}
  }
}
```

Step 2: Retrieve using:
- Semantic search on "machine learning"
- Metadata filter applied

ADVANTAGES:
- Handles complex natural language
- Reduces irrelevant results
- Enables structured search over unstructured queries

ANOLOGY:
"AI papers by Microsoft after 2022"
Parsed to:
query = "AI papers"
filter = {author_org:"Microsoft", year>2022}

## Retrieval Enhancement - Summary Table

| Technique | When to Use | Pros | Cons |
|---|---|---|---|
| **Hybrid Search** | Always in production | Balanced results | More complex |
| **Multi-stage** | High precision needed | Very accurate | Slower, costlier |
| **Recursive** | Multi-hop questions | Deep understanding | Multiple LLM calls |
| **Step-Back** | Reasoning questions | Better conceptual | May over-generalize |
| **Self-Query** | Structured data + NLP | Precise filtering | Requires schema knowledge |

## Metrics - How We Measure Advanced RAG

**Retrieval Metrics**:

| Metric | Formula | What it Measures |
|---|---|---|
| **Recall@k** | #relevant retrieved / #total relevant | Did we find all good chunks? |
| **Precision@k** | #relevant retrieved / k | Are retrieved chunks actually good? |
| **MRR** (Mean Reciprocal Rank) | 1/rank_first_relevant | How early is first good result? |
| **NDCG@k** | Normalized Discounted Cumulative Gain | Ranking quality (position matters) |

# USE CASES OF ARAG?

| Company | Use Case | Naive RAG Problem | Advanced RAG Solution | Company |
|---|---|---|---|---|
| **JPMorgan** | Financial Document Analysis | Missed nuanced regulatory requirements | Query expansion + Multi-stage retrieval | **JPMorgan** |
| **Merck** | Medical Literature Q&A | Retrieved irrelevant drug studies | Reranking + Domain-specific embeddings | **Merck** |
| **Uber** | Customer Support | High latency, irrelevant answers | Hybrid search + Caching | **Uber** |
| **Notion** | AI Q&A over notes | Couldn't handle complex queries | Query rewriting + Recursive retrieval | **Notion** |

WHY BM25?

User Query
↓
BM25 retrieves top 20 chunks
↓
Dense reranker selects top 5
↓
LLM generates answer

# Introduction to Sparse Retrieval?

A family of retrieval methods that represent documents and queries as sparse vectors (mostly zeros) based on term occurrence.

**Key Characteristics:**

- **Vocabulary-based:** Each dimension = a word/term
- **Interpretable:** We know exactly why a document matched
- **Efficient:** Inverted indexes enable fast lookup
- **Precise:** Exact matches are guaranteed

**Analogy:**

Sparse retrieval is like a **library card catalog** - each card lists exactly which books contain which words. To find books about "quantum computing," you look under "quantum" and "computing" cards.

## BM25 - Quick Recap from Previous Class

### BM25 Scoring Formula

$$Score(q, d) = \sum_{t \in q} IDF(t) \times \frac{TF(t, d) \times (k_1 + 1)}{TF(t, d) + k_1 \left(1 - b + b \times \frac{|d|}{avgdl}\right)}$$

**Where**

- $q$= query
- $d$= document
- $t$= query term
- $TF(t, d)$ =term frequency of term $t$in document $d$
- $|d|$= length of the document
- $avgdl$= average document length in the collection
- $k_1$ =term frequency scaling parameter (usually **1.2–2.0**)

### IDF Formula

$$IDF(t) = \log\left(\frac{N - n_t + 0.5}{n_t + 0.5} + 1\right)$$

Where:

- $N$= total number of documents
- $n_t$ =number of documents containing term $t$

## BM25 - The Intuition Behind Parameters

**IDF Formula (BM25)**

$$IDF(t) = \log\left(\frac{N - n_t + 0.5}{n_t + 0.5} + 1\right)$$

**Where:**

- $t$= term
- $N$= total number of documents in the collection
- $n_t$ =number of documents containing term $t$

**Meaning**

- If a term appears in **many documents**, $n_t$ is large → **IDF decreases** (less important term).
- If a term appears in **few documents**, $n_t$ is small → **IDF increases** (more important term).

**Example**

If:

- $N = 1000$documents
- $n_t = 10$documents contain the term

$$IDF(t) = \log\left(\frac{1000 - 10 + 0.5}{10 + 0.5} + 1\right)$$

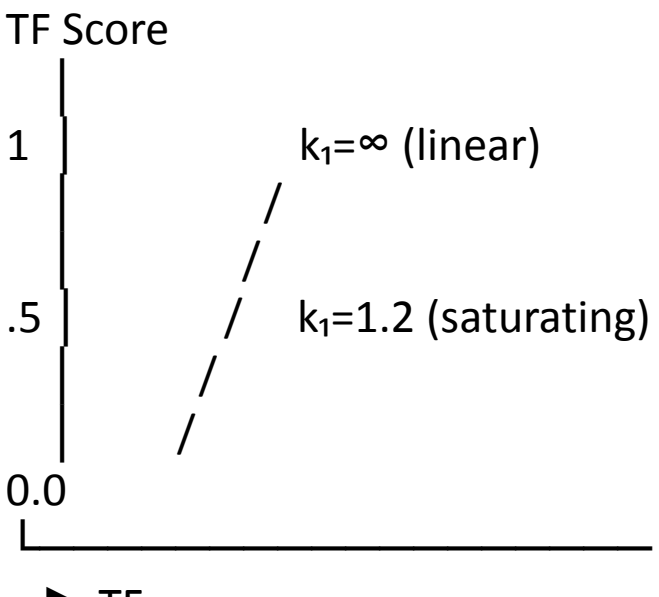
$$IDF(t) \approx \log(95.29) \approx 4.55$$

So the term receives a **high importance weight**.

Effect of $k_1$:

- $k_1 = 0$: Binary model (TF = 0 or 1 only)
- $k_1 \to \infty$: No saturation (TF grows linearly)
- $k_1 = 1.2$-$2.0$: Typical range (saturates after 2-3 occurrences)

Visual Representation:



## The b Parameter (Length Normalization)

Formula Component: $(1 - b + b \times (|d|/avgdl))$

Effect of b:
- b = 0: No length normalization (ignore document length)
- b = 1: Full length normalization (penalize longer docs)
- b = 0.75: Typical default (balanced)

Example Calculation:
avgdl = 100 words
|d| = 200 words (twice average)

b = 0: $1 - 0 + 0 \times 2.0 = 1.0$ (no penalty)
b = 0.75: $1 - 0.75 + 0.75 \times 2.0 = 0.25 + 1.5 = 1.75$ (penalty factor)
b = 1.0: $1 - 1 + 1 \times 2.0 = 2.0$ (max penalty)

TF Score Multiplier = $(k_1 + 1)/(f + k_1 \times \text{length_factor})$
Higher length_factor = Lower TF contribution

**Analogy - Shopping:**
A 1000-page book mentioning "apple" 10 times is less about apples than a 10-page pamphlet mentioning "apple" 5 times. Length normalization captures this.

## BM25 - Complete Numerical Example

Corpus: 1,000 documents
Query: "quantum computing applications"
Document d: 150 words, contains:
    "quantum": 4 times
    "computing": 3 times
    "applications": 2 times
Average document length (avgdl): 120 words
BM25 parameters: $k_1 = 1.5$, $b = 0.75$

**Step 1: Calculate IDF for each term**

Corpus: 1,000 documents
Query: "quantum computing applications"
Document d: 150 words, contains:
"quantum": 4 times
"computing": 3 timesTerm "quantum": appears in 10 documents
IDF(quantum) = log((1000 - 10 + 0.5)/(10 + 0.5))
= log(990.5/10.5)
= log(94.33)
= 4.55

Term "computing": appears in 50 documents
IDF(computing) = log((1000 - 50 + 0.5)/(50 + 0.5))
= log(950.5/50.5)
= log(18.82)
= 2.93

Term "applications": appears in 200 documents
IDF(applications) = log((1000 - 200 + 0.5)/(200 + 0.5))
= log(800.5/200.5)
= log(3.99)
= 1.38
"applications": 2 times

Average document length (avgdl): 120 words
BM25 parameters: $k_1 = 1.5$, $b = 0.75$

## Step 2: Calculate Length Normalization Factor

Document length: $|d| = 150$
Average length: $avgdl = 120$

Length factor $= 1 - b + b \times (|d|/avgdl)$
$= 1 - 0.75 + 0.75 \times (150/120)$
$= 0.25 + 0.75 \times 1.25$
$= 0.25 + 0.9375$
$= 1.1875$

## Step 3: Calculate TF Component for each term:

For "quantum": $f = 4$
$TF(quantum) = (f \times (k_1 + 1)) / (f + k_1 \times length_factor)$
$= (4 \times (1.5 + 1)) / (4 + 1.5 \times 1.1875)$
$= (4 \times 2.5) / (4 + 1.78125)$
$= 10 / 5.78125$
$= 1.73$

For "computing": $f = 3$
$TF(computing) = (3 \times 2.5) / (3 + 1.78125)$
$= 7.5 / 4.78125$
$= 1.57$

For "applications": $f = 2$
$TF(applications) = (2 \times 2.5) / (2 + 1.78125)$
$= 5 / 3.78125$
$= 1.32$

## Step 4: Calculate Final BM25 Score

$$\text{Score}(q,d) = \sum [\text{IDF}(t) \times \text{TF}(t,d)]$$

$$\begin{aligned}\text{Score} &= (4.55 \times 1.73) + (2.93 \times 1.57) + (1.38 \times 1.32) \\ &= 7.87 + 4.60 + 1.82 \\ &= 14.29\end{aligned}$$

**Interpretation:**
**7.87 from "quantum"** (rare term, high weight)
**4.60 from "computing"** (moderately rare)
**1.82 from "applications"** (common term, low weight)

## Scenario 1: Increase "quantum" frequency from 4 to 10

TF(quantum) new = (10 × 2.5) / (10 + 1.78125)
= 25 / 11.78125
= 2.12 (was 1.73)

New contribution = 4.55 × 2.12 = 9.65 (was 7.87)
Total new score = 9.65 + 4.60 + 1.82 = 16.07

Observation: 150% increase in frequency → only 12% increase in score
Reason: TF SATURATION!

## Scenario 2: Document length increases to 300 words

New length factor = 1 - 0.75 + 0.75 × (300/120)
= 0.25 + 0.75 × 2.5
= 0.25 + 1.875
= 2.125

TF(quantum) new = (4 × 2.5) / (4 + 1.5 × 2.125)
= 10 / (4 + 3.1875)
= 10 / 7.1875
= 1.39 (was 1.73)

New quantum contribution = 4.55 × 1.39 = 6.32 (was 7.87)
Total new score = 6.32 + 3.68 + 1.45 = 11.45 (was 14.29)

Observation: Doubling length → 20% score decrease
Reason: LENGTH NORMALIZATION!

## BM25 - Parameter Tuning Guide

### Effect of $k_1$ (Term Frequency Saturation)

| $k_1$ Value | Behavior | Use Case |
|---|---|---|
| 0 - 0.5 | Strong saturation | Short documents, many repeated terms |
| 1.2 - 1.5 | Moderate saturation (default) | General web search |
| 2.0+ | Weak saturation | Long documents, need term frequency signal |

### Effect of b (Length Normalization)

| b Value | Behavior | Use Case |
|---|---|---|
| 0 | No length normalization | All documents same length |
| 0.5 - 0.75 | Moderate normalization (default) | Mixed document lengths |
| 1.0 | Full normalization | Very varied lengths |

Web search:
$k_1 = 1.2$, $b = 0.75$
Academic papers:
$k_1 = 1.5$, $b = 0.5$ (length matters less)
Social media:
$k_1 = 0.5$, $b = 1.0$ (short, varied)

## BM25 vs TF-IDF - Numerical Comparison

TF-IDF Formula: Score = Σ [TF × IDF]
where TF = raw frequency (no saturation)

Term "quantum": TF = 4, IDF = 4.55 → Contribution = 18.2
Term "computing": TF = 3, IDF = 2.93 → Contribution = 8.79
Term "applications": TF = 2, IDF = 1.38 → Contribution = 2.76

TF-IDF Score = 18.2 + 8.79 + 2.76 = 29.75

BM25 Score (from earlier) = 14.29

| Term | Frequency | TF-IDF Weight | BM25 Weight | Why Different? |
|---|---|---|---|---|
| quantum | 4 | 18.2 | 7.87 | BM25 saturates high frequencies |
| computing | 3 | 8.79 | 4.60 | BM25 saturates |
| applications | 2 | 2.76 | 1.82 | BM25 saturates |
| **TOTAL** | | **29.75** | **14.29** | BM25 more conservative |

## BM25 with Query Term Weighting

**Advanced: Queries have term weights too**

### Extended BM25 Scoring Formula

$$Score(q, d) = \sum_{t \in q} IDF(t) \times \frac{TF(t, d) \times (k_1 + 1)}{TF(t, d) + k_1 \left(1 - b + b \times \frac{|d|}{avgdl}\right)} \times \frac{(k_3 + 1)\, qtf}{k_3 + qtf}$$

**Where:**

- $q$= query
- $d$= document
- $t$= query term
- $TF(t, d)$ =term frequency of term $t$in document $d$
- $qtf$= term frequency in the **query**
- $IDF(t)$ =inverse document frequency of term $t$
- $|\ d\ |$= document length
- $avgdl$= average document length in the collection
- $k_1$ =term frequency scaling parameter (typically **1.2 – 2.0**)
- $k_3$ =query term frequency parameter
- $b$= document length normalization parameter (**0.75 commonly**)

### IDF Formula

$$IDF(t) = \log\left(\frac{N - n_t + 0.5}{n_t + 0.5} + 1\right)$$

Where:

- $N$= total number of documents
- $n_t$ =number of documents containing term $t$

## BM25 with Query Term Weighting

## Advanced: Queries have term weights too

**Query**

"quantum computing quantum algorithms"

The term **"quantum" appears twice** in the query.

So,

$$qtf = 2$$

**Step 1: Query Term Frequency Factor**

The query factor in **Extended BM25** is:

$$\frac{(k_3 + 1) \times qtf}{k_3 + qtf}$$

**Case 1: Large $k_3$)Typical Implementation)**

Let

$$k_3 = 1000$$

$$\frac{(1000 + 1) \times 2}{1000 + 2} = \frac{2002}{1002} \approx 1.998 \sim 2$$

**Case 2: Smaller $k_3$**

Let

$$k_3 = 1.2$$

$$\frac{(1.2 + 1) \times 2}{1.2 + 2} = \frac{2.2 \times 2}{3.2} = 1.375$$

## BM25 with Query Term Weighting

**Advanced: Queries have term weights too**

### Step 2: Document Scoring Example

| Parameter | Value |
|---|---|
| IDF(quantum) | 4.55 |
| TF normalization | 1.73 |
| Query factor | 1.375 |

**Without Query Frequency Factor**

$$Score = 4.55 \times 1.73$$
$$Score = 7.87$$

**With Query Frequency Factor**

$$Score = 4.55 \times 1.73 \times 1.375$$
$$Score = 10.82$$

- When **query terms repeat**, BM25 can increase their importance.
- **Large** $k_3$ →query frequency effect is minimal.
- **Smaller** $k_3$ →repeated terms influence ranking more.

"quantum" appears twice in query (qtf = 2)
With $k_3$ = 1000 (large): factor ≈ 1 (qtf ≈ 1 effectively)
With $k_3$ = 1.2: factor = (1.2 + 1) × 2 / (1.2 + 2) = 2.2 × 2 / 3.2 = 1.375
Quantum contribution becomes: 4.55 × 1.73 × 1.375 = 10.82 (was 7.87)

## BM25 in Production – Inverted Index

- In real-world search systems, the **BM25 ranking algorithm** is implemented using an **inverted index** to efficiently retrieve documents that contain query terms.
- Instead of scanning every document in the collection, the inverted index stores a mapping from terms to the documents in which they appear, along with their frequencies.
- This structure allows BM25 to quickly compute relevance scores.

**Inverted Index Structure**

An inverted index typically contains:

| Term | Posting List |
|---|---|
| quantum | (Doc1, TF=3), (Doc4, TF=1), (Doc8, TF=2) |
| computing | (Doc2, TF=1), (Doc4, TF=2) |
| algorithm | (Doc3, TF=2), (Doc8, TF=1) |

Each **posting list** stores:

- Document ID
- Term frequency in that document
- Optional metadata such as document length or positions

**BM25 Retrieval Process in Production**

1. **Query Processing:**
   The user query is tokenized and normalized.
2. **Posting List Lookup:**
   The inverted index retrieves the list of documents containing each query term.
3. **Score Calculation:**
   BM25 calculates a relevance score for each document using term frequency, document length normalization, and IDF.
4. **Ranking:**
   Documents are ranked based on BM25 scores.
5. **Top-k Retrieval:**
   Only the top-ranked documents are returned to the user or passed to downstream models (e.g., RAG systems).

**Why Inverted Index is Used**

- Enables **fast retrieval even with millions of documents**
- Reduces **search complexity**
- Supports **efficient BM25 scoring**
- Used in large-scale systems such as **search engines and information retrieval systems**

# GenerativeAI and Its Applications

## BM25 Variants and Extensions

BM25 (Original)

- BM25F
  - BM25 with multiple document fields
  - (Different weights for title, abstract, body)
- BM25+
  - Improves the scoring formula
  - Prevents negative scores and improves term saturation
- BM25L
  - Designed for long documents
  - Reduces length normalization bias
- BM25-adpt
  - Adaptive BM25
  - Automatically tunes parameters for each term

ANOLOGY:

| Field | Content | Weight |
|---|---|---|
| Title | "Quantum Computing Breakthrough" | 3.0 |
| Abstract | "... quantum computing applications ..." | 1.5 |
| Body | Full document text | 1.0 |

**Scoring Formula**

$$Score = 3.0 \times BM25(title) + 1.5 \times BM25(abstract) + 1.0 \times BM25(body)$$

BM25 for RAG – Best Practices

1. **Chunk Size Matters:**

Recommended chunk size:

**100 – 500 words**

| Too Small | Too Large |
|---|---|
| Context lost | Multiple topics mixed |
| Weak IDF signal | Relevance diluted |

**2. Field Boosting**

Example Index Structure

```
chunk_title   → weight = 3.0
chunk_heading → weight = 2.0
chunk_text    → weight = 1.0
```

Example Query:

```
Query: "quantum computing"
```

Match in **title** → Higher BM25 score.

**3. Query Expansion**

Original query: AI ethics

Expanded query

"AI ethics"

OR

"artificial intelligence ethics"

OR

"AI moral responsibility"

BM25 naturally handles **OR queries** by **summing term scores**.

**4. Hybrid Retrieval**

Most modern RAG systems combine **BM25 + dense embeddings**

$$Final\ Score = \alpha \times BM25 + (1 - \alpha) \times Dense$$

Typical values:

$\alpha = 0.3 - 0.5$

## BM25 Limitations and Alternatives

| Limitation | Example | Solution |
|---|---|---|
| Vocabulary mismatch | "car" vs "automobile" | Dense retrieval |
| Weak semantic understanding | "Apple" fruit vs company | Contextual embeddings |
| Rare new terminology | New AI terms | Query expansion |
| Stop words ignored | "the", "is" | Stop word removal |
| Multi-word expressions | "New York" | Phrase indexing |

Example: LangChain BM25 Retriever

```
from langchain.retrievers import BM25Retriever
from langchain.schema import Document

documents = [
    Document(page_content="Quantum computing uses qubits"),
    Document(page_content="Applications of quantum computing"),
]

bm25_retriever = BM25Retriever.from_documents(documents)
bm25_retriever.k = 5

results = bm25_retriever.get_relevant_documents(
    "quantum computing applications"
)
```

Example: Hybrid Search (Vector + Keyword)

```
from llama_index import VectorStoreIndex
from llama_index.retrievers import RouterRetriever

vector_retriever = VectorStoreIndex.from_documents(documents).as_retriever()
keyword_retriever = BM25Retriever.from_documents(documents)

hybrid_retriever = RouterRetriever.from_retrievers(
    [vector_retriever, keyword_retriever],
    selection_mode="weighted"
)
```

BM25 Numerical Exercise

**Problem:**
Given a corpus of 500 documents, query = "neural networks training", document d has:

- Length = 200 words
- Contains "neural": 5 times (appears in 20 docs)
- Contains "networks": 4 times (appears in 25 docs)
- Contains "training": 3 times (appears in 100 docs)
- Average document length = 150 words
- $k_1$ = 1.2, b = 0.75

**Calculate:**

1. IDF for each term
2. Length normalization factor
3. TF component for each term
4. Final BM25 score

BM25 Numerical Exercise

**Step 1: IDF Calculation**

$$IDF(neural) = log\big((500 - 20 + 0.5)/(20 + 0.5)\big)$$
$$= log(23.44) = 3.15$$
$$IDF(networks) = log(18.65) = 2.93$$
$$IDF(training) = log(3.98) = 1.38$$

**Step 2: Length Normalization**

$$Length\ factor = 1 - b + b \times \frac{|\ d\ |}{avgdl}$$
$$= 0.25 + 0.75 \times (200/150)$$
$$= 1.25$$

**Step 3: TF Components**

$$TF(neural) = \frac{5 \times 2.2}{5 + 1.2 \times 1.25} = 1.69$$
$$TF(networks) = 1.60$$
$$TF(training) = 1.47$$

**Step 4: Final BM25 Score**

$$Score$$
$$= 3.15 \times 1.69 + 2.93 \times 1.60 + 1.38 \times 1.47$$
$$Score = 5.32 + 4.69 + 2.03$$
$$Final\ Score = 12.04$$

# THANK YOU